

Indexing & performance basics

COMPLETE LEARNING GUIDE

What You'll Learn

An index is the single highest-leverage tool for making a slow query fast, and also one of the easiest things to misuse by over-applying — every index added speeds up certain reads while adding overhead to every write. This guide explains how the B-tree index structure that underlies most database indexes actually works, the difference between a clustered primary key and secondary indexes, when an index genuinely helps a query (and the common cases where it silently doesn't, even though it exists), how column order in a composite index changes which queries it can serve, covering indexes that avoid a table lookup entirely, and the ongoing cost indexes impose on every write — closing with a practical process for actually diagnosing a slow query rather than guessing.

1. How B-Tree Indexes Work

Most general-purpose database indexes are B-tree (balanced tree) structures: a sorted, hierarchical structure that lets the database find a specific value, or a range of values, in roughly logarithmic time instead of scanning every row. Without an index, finding a row matching a WHERE condition requires a sequential scan — reading every single row in the table to check if it matches, which scales linearly with table size and becomes painfully slow on large tables. An index on the filtered column lets the database jump almost directly to the matching rows instead.

2. Primary Keys and Clustered Indexes

A primary key is automatically indexed and, in many database engines (including PostgreSQL's default table structure and MySQL's InnoDB), determines the physical or logical ordering the table's data is stored in — a clustered index. Every additional index you create beyond the primary key is a secondary index: a separate structure mapping indexed column values to the location of the actual row, meaning a lookup through a secondary index typically involves an extra step (finding the row's location, then fetching it) compared to a lookup that can be satisfied by the clustered index alone.

3. When an Index Helps (and When It Doesn't)

An index helps queries that filter (WHERE), join (ON), or sort (ORDER BY) on the indexed column(s) — assuming the query is selective enough that using the index is actually cheaper than scanning the table (for a column where most rows match, like a boolean with 90% true values, a sequential scan can be faster than random-access index lookups for that many rows). An index does not help when a function or expression wraps the indexed column in the query (WHERE LOWER(email) = ... won't use a plain index on email), when the query filters on a different column than what's indexed, or on very small tables where a full scan is already essentially instant.

SQL

```
-- This index won't be used because LOWER() wraps the column:
SELECT * FROM students WHERE LOWER(email) = 'aisha@example.com';

-- Fix: index the expression itself
CREATE INDEX idx_students_email_lower ON students (LOWER(email));
```

4. Composite Indexes and Column Order

A composite (multi-column) index is built as a single sorted structure across multiple columns in a specific declared order, and that order determines which queries it can actually serve — it behaves like a phone book sorted by last name then first name: you can efficiently find 'Smith', or 'Smith, John', but you cannot efficiently find everyone with first name 'John' regardless of last name using that same structure. The rule of thumb: put the column used for equality filtering first, and the column used for range filtering or sorting last — a composite index on (status, created_at) efficiently serves WHERE status = 'active' ORDER BY created_at, but a query filtering only on created_at can't use it at all.

SQL

```
CREATE INDEX idx_enrollments_student_status ON enrollments (student_id, status);

-- Uses the index efficiently (matches column order):
SELECT * FROM enrollments WHERE student_id = 42 AND status = 'active';

-- Cannot use this index efficiently (skips the leading column):
SELECT * FROM enrollments WHERE status = 'active';
```

5. Covering Indexes

A covering index includes every column a query needs — both the filtered/sorted columns and the columns being selected — so the database can satisfy the entire query directly from the index itself, without a second lookup back to the actual table row (sometimes called avoiding a 'bookmark lookup' or achieving an 'index-only scan'). This is a meaningful performance win for hot, frequently-run queries, at the cost of a larger index that duplicates more column data and adds more write overhead — making it a deliberate optimization for specific known-hot queries, not a default to apply everywhere.

6. Index Maintenance Cost on Writes

Every index on a table must be updated on every INSERT, UPDATE (of an indexed column), and DELETE — meaning each additional index makes writes slightly slower and consumes additional storage, permanently. A table with fifteen indexes because every column that might ever be filtered on got one is a very common real-world performance anti-pattern: it looks harmless in read-focused testing and quietly cripples write throughput in production. The right number of indexes is driven by actual, observed query patterns — not by indexing every column defensively.

7. Diagnosing a Slow Query

The reliable process is: identify the actual slow query (via slow query logs or application-level timing, not guessing), run EXPLAIN ANALYZE on it to see the real execution plan and where time is actually being spent, check whether a sequential scan is happening where an index lookup was expected, and check whether an existing index isn't being used because of a wrapped column, a mismatched composite index column order, or low selectivity. Adding an index should be a response to this kind of concrete diagnosis, not a first reflex applied to every column that appears in a WHERE clause somewhere in the codebase.

Common Pitfalls

- Adding an index to every column 'just in case,' which slows down every write without meaningfully helping reads that never filter or sort on that column.
- Wrapping an indexed column in a function in the query (LOWER(), DATE(), etc.) without an expression index, silently preventing the existing index from being used.
- Getting composite index column order wrong — putting a range-filtered or rarely-used column first, making the index unusable for the query's actual equality filter.
- Indexing a low-cardinality column (like a boolean or a status with only two or three values) and expecting a large performance gain that a sequential scan often matches anyway.
- Never revisiting index usage over time — accumulating unused indexes from queries that were removed or changed long ago, still silently taxing every write.
- Assuming an index fixes any slow query regardless of the query's actual shape, instead of running EXPLAIN to check what's really happening.

Key Takeaways

- Indexes trade faster reads for slower writes and more storage — they are not a free performance upgrade to apply everywhere.
- A composite index's column order determines which queries it can serve — equality columns first, range/sort columns last is the standard heuristic.
- Wrapping an indexed column in a function in a WHERE clause defeats a plain index unless an expression index exists for that exact expression.

- EXPLAIN ANALYZE is how you verify whether an index is actually being used, rather than assuming it is because it exists.
- Low-cardinality columns often don't benefit meaningfully from indexing, since a sequential scan can be just as fast when most rows match anyway.
- Let real, observed query patterns drive which indexes exist — not defensive indexing of every column that could theoretically be filtered on.

Learn More

PostgreSQL Docs — Indexes

<https://www.postgresql.org/docs/current/indexes.html>

Use The Index, Luke — SQL Indexing Guide

<https://use-the-index-luke.com/>

PostgreSQL Docs — Multicolumn Indexes

<https://www.postgresql.org/docs/current/indexes-multicolumn.html>